The slow speed of I/O compared to a typical CPU clock cycle shows that management of CPU usage is vital to ensure that the CPU does not remain idle while I/O is taking place. This is discussed in the next section.

20.03 Process scheduling

Resource management relating to the CPU mainly concerns scheduling to ensure efficient usage. The methods discussed here consider the CPU as a single unit; specific issues relating to a multiprocessor system are not considered.

Programs that are available to be run on a computer system are initially stored on disk. A user could submit a program as a 'job' which would include the program and some instructions about how it should be run. Figure 20.04 shows an overview of the components involved when a program is run.

A long-term or **high-level scheduler** program controls the selection of a program stored on disk to be moved into main memory. Occasionally a program has to be taken back to disk due to the memory getting overcrowded. This is controlled by a medium-term scheduler. When the program is installed in memory, a short-term or **low-level scheduler** controls when it has access to the CPU.

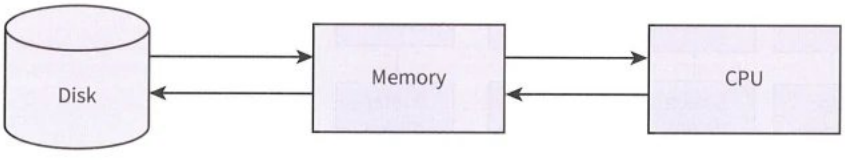


Figure 20.04 Components involved in running a program

KEY TERMS

High-level scheduler: makes decisions about which program stored on disk should be moved into memory

Low-level scheduler: makes decisions about which process stored in memory should have access to the CPU

Process states

In Chapter 8 (Section 8.02), it was stated that a **process** can be defined as 'a program being executed'. This definition can be improved by including the state when the program first arrives in memory. At this stage a **process control block (PCB)** can be created in memory ready to receive data when the process is executed. Once in memory the state of the process can change.

The transitions between the states shown in Figure 20.05 can be described as follows.

- A new process arrives in memory and a PCB is created; it changes to the ready state.
- A process in the ready state is given access to the CPU by the dispatcher; it changes to the running state.
- A process in the running state is halted by an interrupt; it returns to the ready state.
- A process in the running state cannot progress until some event has occurred (I/O perhaps); it changes to the waiting state (sometimes called the 'suspended' or 'blocked' state).

- A process in the waiting state is notified that an event is completed; it returns to the ready state.
- A process in the running state completes execution; it changes to the terminated state.

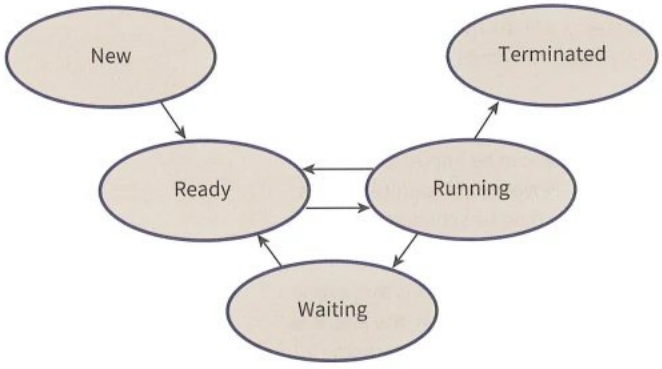


Figure 20.05 The five states defined for a process being executed

It is possible for a process to be separated into different parts for execution. The separate parts are called **threads**. If this has happened, each thread is handled as an individual process.

KEY TERMS

Process: a program in memory that has an associated process control block

Process control block (PCB): a complex data structure containing all data relevant to the running of a process

Thread: part of a program which is handled as an individual process when being executed

Interrupts

Some interrupts are caused by errors that prematurely terminate a running process. Otherwise there are two reasons for interrupts.

- Processes consist of alternating periods of CPU usage and I/O usage. I/O takes far too long for the CPU to remain idle waiting for it to complete. The interrupt mechanism is used when a process in the running state makes a system call requiring an I/O operation and has to change to the waiting state.
- The scheduler decides to halt the process for one of several reasons as discussed later under the heading 'Scheduling algorithms'.

Whatever the reason for an interrupt, the OS kernel must invoke an interrupt-handling routine. This may have to decide on the priority of an interrupt. One required action is that the current values stored in registers must be recorded in the process control block. This allows the process to continue execution when it eventually returns to the running state.

Discussion Point:

What would happen if an interrupt was received while the interrupt-handling routine was being executed by the CPU? Does this require a priority being set for each interrupt?

Scheduling algorithms

Although the long-term or high-level scheduler will have decisions to make when choosing which program should be loaded into memory, we concentrate here on the options for the short-term or low-level scheduler.

A scheduling algorithm can be preemptive or non-preemptive. A preemptive algorithm can halt a process that would otherwise continue running undisturbed. If an algorithm is preemptive it may involve prioritising processes.

The simplest possible algorithm is first come first served (FCFS). This is a non-preemptive algorithm and can be implemented by placing the processes in a first-in first-out (FIFO) queue. The algorithm will be very inefficient if it is the only algorithm employed but it can be used as part of a more complex algorithm.

A round-robin algorithm allocates a time slice to each process and is therefore preemptive, because a process will be halted when its time slice has run out. It can be implemented as a FIFO queue. It normally does not involve prioritising processes. However, if separate queues are created for processes of different priorities then each queue could be scheduled using a round-robin algorithm.

A priority-based scheduling algorithm is more complicated. One reason for this is that every time a new process enters the ready queue or when a running process is halted, the priorities for the processes may have to be re-evaluated. The other reason is that whatever scheme is used to judge priority level it will require some computation. Possible criteria are:

- estimated time of process execution
- estimated remaining time for execution
- length of time already spent in the ready queue
- whether the process is I/O bound or CPU bound.

More than one of these criteria might be considered. Clearly, estimating a time for execution may not be easy. Some processes require extensive I/O, for instance printing wage slips for employees. There is very little CPU usage for such a process so it makes sense to allocate it a high priority so that the small amount of CPU usage can take place. The process will then change to the waiting state while the printing takes place.

20.04 Memory management

The term memory management includes a number of aspects.

- The provision of protected memory space for the OS kernel.
- The loading of a program into memory requires defining the memory addresses for the program itself, for associated procedures and for the data required by the program.
- In a multiprogramming system, this might not be straightforward. The storage of processes in main memory can get fragmented in the same way as happens for files stored on a hard disk. There may be a need for the medium-term scheduler to move a process out of main memory to ease the problem.
- Decisions have to be made about how large a part of memory should be allocated to individual processes sharing memory.

Partitions and segments

An early approach to memory management when different processes were loaded into memory simultaneously was to partition the memory. The aim was to load the whole of a process into one partition. This was wasteful of memory if the process size was less than the partition size. An improvement was dynamic partitioning where the partition size was allowed to adjust to match the process size. The rule of one process per partition remained.